

Homework #1 – Neural Network Overview – Activation Functions and Back Propagation

CAP4613/CAP 5619, Deep & Reinforcement Learning (Fall 2026), Department of Computer Science, Florida State University

Points: 60

Due: 11:59pm on Friday, September 18th, 2026

Submission: You need to submit electronically via Canvas by uploading separately a) a pdf file (named “hw1-Firstname-Lastname.pdf”) for your answers to the questions, and b) the program(s) you have created (named as “hw1-prog-Firstname-Lastname.???”); if there are multiple program files, please zip them as a single archive. Here replace “Firstname” using your first name and replace “Lastname” using your last name in the file names. Note that you must upload the pdf file for your answers as a **separate file** and do not include the pdf in your archive file.

The main purpose of this assignment is to let you be familiar with neural network architecture elements including activation functions and how to define and train a simple neural network.

Problem 1 (30 points, 5 point each) As neural networks are typically trained using (stochastic) gradient descent optimization algorithms, properties of the activation functions affect the learning. Here we divide the domain of an activation function into: 1) **fast learning region** if the magnitude of the gradient is larger than 0.99, 2) **active learning region** if the magnitude of the gradient is between 0.01 and 0.99 (inclusive), 3) **slow learning region** if the magnitude of the gradient is larger than 0 but smaller than 0.01, and 4) **inactive learning region** if the magnitude of the gradient is 0. For each of the following activation functions, **plot** its gradient in the range from -5 to 5 of the input and then **list** the four types of regions. If the gradient is not well defined for an input value, indicate so and then use any reasonable value.

(1) Rectified linear unit $f(z) = \begin{cases} z & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$.

(2) Scaled logistic sigmoid activation function $f(z) = \sigma'(z) = \frac{1}{1+e^{-z/0.2}}$.

(3) Piece-wise linear unit $f(z) = \begin{cases} 0.4z + 0.6 & \text{if } z > 1 \\ z & \text{if } 1 \geq z \geq -1 \\ 0.4z - 0.6 & \text{Otherwise} \end{cases}$.

(4) Swish $f(z) = z\sigma(3z)$, where $\sigma(z) = \frac{1}{1+e^{-z}}$.

(5) Exponential Linear Unit (ELU) $f(z) = \begin{cases} z & \text{if } z \geq 0 \\ 0.2(e^z - 1) & \text{otherwise} \end{cases}$. (Note here is a special case of the general ELU function with $\alpha=0.2$.)

(6) Gaussian Error Linear Unit (GELU), $f(x) = \frac{x}{2} \left(1 + \operatorname{erf} \left(\frac{x}{\sqrt{2}} \right) \right)$, where erf is the error function, (also known as the Gauss error function), given by $\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$. (Note that there is an approximation for GELU using $x\sigma(1.702x)$, which you can use; in addition, there are also other approximations for the error function you could use. However, typical deep learning frameworks provide an efficient implementation of the error function and GELU.)

Problem 2 (15 points) We use a modified subset of the well-known Iris dataset (see <https://archive.ics.uci.edu/dataset/53/iris>); the dataset file name is "iris_data_subset.txt", where the first two numbers are features and the last one class label for each row (there are twenty rows in total).

- 1) **(5 points)** Visualize the data points and then design a neural network that classifies all the data points correctly.
- 2) **(10 points)** Implement your designed neural network in the deep learning framework of your choice and verify all the samples are classified correctly.

Problem 3 (15 points). As an exercise to gain better understanding of back propagation, swap the label for each sample and your designed network now should give a classification accuracy of 0%. Answer the following questions:

- 1) **(5 points)** Inspect the neural network and weights provided in "hw1_problem3.ipynb," calculate the gradient for each of all 13 trainable parameters for the first data sample in the dataset **before any parameter update**. Use the **binary cross-entropy loss** $l = -y \log p - (1 - y) \log(1 - p)$, where y is the swapped label and p is the network's predicted probability of class 1. You need to show the intermediate steps.
- 2) **(10 points)** Use your deep learning framework to retrain your designed network so that it can classify all the samples correctly again. You need to specify the hyperparameters you have used (such as learning rate) and the loss function; you also need to plot how the loss and classification accuracy evolve over the training epochs.